

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.	BL2 – Understand
2	Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level:	BL2 – Understand
3	Correct a faulty SELECT query where column names are misspelled or improperly referenced.	BL1 – Remember
4	Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.	BL3 – Apply
5	Identify and correct a subquery error where multiple values are returned instead of a single value.	BL2 – Understand
6	Debug a VIEW creation statement that contains invalid column references or syntax errors.	BL1 – Remember
7	Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.	BL3 – Apply
8	Identify violations of integrity constraints in a given table and correct them.	BL2 – Understand

9	Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.	BL3 – Apply
10	Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.	BL3 – Apply

Code of Conduct:

I VENNA MEENAKSHI REDDY (192471048) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: VENNA MEENAKSHI REDDY

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	Produces clean, well-structured code with proper comments and	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

	documentatio n			
--	-------------------	--	--	--

1. Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.

Incorrect SQL Statement

```
CREATE TABLE Student
(  
    StudentID INT,  
    Name VARCHAR(50)  
    Age INT,  
    Email VARCHAR(100),  
    PRIMARY StudentID  
);
```

Errors

1. Missing comma after Name VARCHAR(50).
2. Incorrect PRIMARY keyword; it should be PRIMARY KEY.
3. StudentID should typically be NOT NULL because it is the primary key.
4. Email can be made UNIQUE to prevent duplicate email addresses.

Correct SQL Statement

```
CREATE TABLE Student
(  
    StudentID INT NOT NULL,  
    Name VARCHAR(50),  
    Age INT,  
    Email VARCHAR(100) UNIQUE,  
    PRIMARY KEY (StudentID)  
);
```

Constraints Added:

- **NOT NULL:** Ensures StudentID cannot contain NULL.
- **PRIMARY KEY:** Uniquely identifies each student.
- **UNIQUE:** Prevents duplicate email addresses.

Output

```
MySQL 8.0 Command Line CLI x + v
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use dbms;
Database changed
mysql> CREATE TABLE Student
-> (
->     StudentID INT NOT NULL,
->     Name VARCHAR(50),
->     Age INT,
->     Email VARCHAR(100) UNIQUE,
->     PRIMARY KEY (StudentID)
-> );
Query OK, 0 rows affected (0.09 sec)

mysql> DESC Student;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| StudentID | int | NO | PRI | NULL | |
| Name | varchar(50) | YES | | NULL | |
| Age | int | YES | | NULL | |
| Email | varchar(100) | YES | UNI | NULL | |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.03 sec)

mysql> |
```

This corrected statement is syntactically valid and includes appropriate constraints for maintaining data integrity.

2. Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue.

Incorrect SQL Query

```
SELECT Department, AVG(Salary)
FROM Employee;
```

Error

- The query uses the aggregate function AVG(Salary) along with the non-aggregated column Department.
- Since Department is not included in a GROUP BY clause, most SQL databases will return an error.

Correct SQL Query

```
SELECT Department, AVG(Salary) AS AverageSalary
FROM Employee
GROUP BY Department;
```

Explanation

- AVG(Salary) calculates the average salary.
- GROUP BY Department groups employees by department.
- AVG(Salary) is then calculated separately for each department.

Example

Employee Table

EmployeeID	Name	Department	Salary
101	John	HR	30000
102	Alice	HR	40000
103	Bob	IT	50000
104	Carol	IT	60000

Query

```
SELECT Department, AVG(Salary) AS AverageSalary
FROM Employee
GROUP BY Department;
```

Output

```
MySQL 8.0 Command Line Cli  ×  +  ▾

Database changed
mysql> CREATE TABLE Employee (
  ->   EmployeeID INT PRIMARY KEY,
  ->   Name VARCHAR(50),
  ->   Department VARCHAR(30),
  ->   Salary DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Employee (EmployeeID, Name, Department, Salary)
  -> VALUES
  -> (101, 'John', 'HR', 30000),
  -> (102, 'Alice', 'HR', 40000),
  -> (103, 'Bob', 'IT', 50000),
  -> (104, 'Carol', 'IT', 60000);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> select *from employee;
+-----+-----+-----+-----+
| EmployeeID | Name | Department | Salary |
+-----+-----+-----+-----+
| 101 | John | HR | 30000.00 |
| 102 | Alice | HR | 40000.00 |
| 103 | Bob | IT | 50000.00 |
| 104 | Carol | IT | 60000.00 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT Department, AVG(Salary) AS AverageSalary
  -> FROM Employee
  -> GROUP BY Department;
+-----+-----+
| Department | AverageSalary |
+-----+-----+
| HR | 35000.000000 |
| IT | 55000.000000 |
+-----+-----+
2 rows in set (0.00 sec)

mysql> |
```

This corrected query executes successfully and returns the average salary for each department.

3. Correct a faulty SELECT query where column names are misspelled or improperly referenced.

Incorrect Query

```
SELECT StudntID, Nmae, Age
FROM Student;
```

Errors

1. StudntID is misspelled (should be StudentID).
2. Nmae is misspelled (should be Name).

Correct Query

```
SELECT StudentID, Name, Age
FROM Student;
```

Another Example (Incorrect Table Alias)

Incorrect Query

```
SELECT S.StudentID, S.Name
FROM Student;
```

Error

- The alias S is used but not defined.

Correct Query

```
SELECT S.StudentID, S.Name
FROM Student AS S;
```

Output

```
mysql> CREATE TABLE Student (
->     StudentID INT PRIMARY KEY,
->     Name VARCHAR(50),
->     Age INT,
->     Email VARCHAR(100)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Student (StudentID, Name, Age, Email)
-> VALUES
-> (101, 'John', 20, 'john@example.com'),
-> (102, 'Alice', 21, 'alice@example.com'),
-> (103, 'Bob', 22, 'bob@example.com');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT StudentID, Name, Age
-> FROM Student;
+-----+-----+-----+
| StudentID | Name  | Age  |
+-----+-----+-----+
| 101      | John  | 20   |
| 102      | Alice | 21   |
| 103      | Bob   | 22   |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> |
```


4. Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.

Incorrect SQL Query

```
SELECT Employee.Name, Department.DepartmentName
FROM Employee
JOIN Department;
```

Errors

1. The JOIN statement is missing the ON clause.
2. Without a join condition, the database cannot determine how the two tables are related (or it may produce a Cartesian product in some databases).

Correct SQL Query

```
SELECT Employee.Name, Department.DepartmentName
FROM Employee
JOIN Department
ON Employee.DepartmentID = Department.DepartmentID;
```

Explanation

- JOIN combines rows from two tables.
- The ON clause specifies the matching columns between the tables.
- Employee.DepartmentID = Department.DepartmentID ensures that each employee is matched with the correct department.

OUTPUT

```
mysql> CREATE TABLE Department (
->   DepartmentID INT PRIMARY KEY,
->   DepartmentName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> CREATE TABLE Employee (
->   EmployeeID INT PRIMARY KEY,
->   Name VARCHAR(50),
->   DepartmentID INT
-> );
Query OK, 0 rows affected (0.10 sec)

mysql> INSERT INTO Department VALUES
-> (1, 'HR'),
-> (2, 'IT');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO Employee VALUES
-> (101, 'John', 1),
-> (102, 'Alice', 2),
-> (103, 'Bob', 1);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT Employee.Name, Department.DepartmentName
-> FROM Employee
-> JOIN Department
-> ON Employee.DepartmentID = Department.DepartmentID;
+-----+-----+
| Name | DepartmentName |
+-----+-----+
| John | HR              |
| Alice| IT              |
| Bob  | HR              |
+-----+-----+
3 rows in set (0.00 sec)

mysql> |
```

5. Identify and correct a subquery error where multiple values are returned instead of a single value.

Incorrect SQL Query

```
SELECT Name
FROM Employee
WHERE DepartmentID = (
    SELECT DepartmentID
    FROM Department
);
```

Errors

1. The subquery returns multiple values, but the = operator expects only a single value.
2. This results in an error such as "Subquery returns more than one row."

Correct SQL Query

```
SELECT Name
FROM Employee
WHERE DepartmentID IN (
    SELECT DepartmentID
    FROM Department
);
```

Explanation

- Use the IN operator when the subquery returns multiple values.
- IN compares the value with all values returned by the subquery, avoiding the error.
- Use = only when the subquery is guaranteed to return exactly one value.

Output:

```
MySQL 8.0 Command Line Cll x + v
mysql> use dbms;
Database changed
mysql> CREATE TABLE Department (
  ->   DepartmentID INT PRIMARY KEY,
  ->   DepartmentName VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE Employee (
  ->   EmployeeID INT PRIMARY KEY,
  ->   Name VARCHAR(50),
  ->   DepartmentID INT
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Department VALUES
  -> (1, 'HR'),
  -> (2, 'IT'),
  -> (3, 'Sales');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Employee VALUES
  -> (101, 'John', 1),
  -> (102, 'Alice', 2),
  -> (103, 'Bob', 3),
  -> (104, 'David', 2);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT Name
  -> FROM Employee
  -> WHERE DepartmentID = (
  ->   SELECT DepartmentID
  ->   FROM Department
  -> );
ERROR 1242 (21000): Subquery returns more than 1 row
mysql> SELECT Name
  -> FROM Employee
  -> WHERE DepartmentID IN (
  ->   SELECT DepartmentID
  ->   FROM Department
  -> );
+-----+
| Name |
+-----+
| John |
| Alice |
| Bob |
| David |
+-----+
4 rows in set (0.00 sec)

mysql> |
```

6. Debug a VIEW creation statement that contains invalid column references or syntax errors.

Incorrect SQL Query

```
CREATE VIEW Student_View AS
SELECT StudentID, FullName, Age
FROM Student;
```

Errors

1. FullName does not exist in the Student table.
2. The correct column name is Name.

Correct SQL Query

```
CREATE VIEW Student_View AS
SELECT StudentID, Name, Age
FROM Student;
```

Explanation

- A VIEW is a virtual table based on an SQL query.
- All referenced columns must exist in the base table.

Output

```
mysql> CREATE TABLE Student (
  ->     StudentID INT PRIMARY KEY,
  ->     Name VARCHAR(50),
  ->     Age INT
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO Student VALUES
  -> (101, 'Rahul', 20),
  -> (102, 'Priya', 21),
  -> (103, 'Amit', 19);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> CREATE VIEW Student_View AS
  -> SELECT StudentID, FullName, Age
  -> FROM Student;
ERROR 1054 (42S22): Unknown column 'FullName' in 'field list'
mysql> CREATE VIEW Student_View AS
  -> SELECT StudentID, Name, Age
  -> FROM Student;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Student_View;
+-----+-----+-----+
| StudentID | Name | Age |
+-----+-----+-----+
| 101 | Rahul | 20 |
| 102 | Priya | 21 |
| 103 | Amit | 19 |
+-----+-----+-----+
3 rows in set (0.01 sec)

mysql> |
```

7. Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.

Incorrect SQL Query

```
SELECT *  
FROM Student  
WHERE Age > 18 AND Age > 15;
```

Error

- Age > 15 is unnecessary because Age > 18 already satisfies it.

Correct SQL Query

```
SELECT *  
FROM Student  
WHERE Age > 18;
```

Explanation

- Removing redundant conditions improves query readability and efficiency.

Output

```
mysql> CREATE TABLE Student (  
->     StudentID INT PRIMARY KEY,  
->     Name VARCHAR(50),  
->     Age INT  
-> );  
Query OK, 0 rows affected (0.05 sec)  
  
mysql> INSERT INTO Student VALUES  
-> (101, 'Rahul', 20),  
-> (102, 'Priya', 17),  
-> (103, 'Amit', 19),  
-> (104, 'Sneha', 22),  
-> (105, 'Kiran', 15);  
Query OK, 5 rows affected (0.01 sec)  
Records: 5  Duplicates: 0  Warnings: 0  
  
mysql> SELECT *  
-> FROM Student  
-> WHERE Age > 18;  
+-----+-----+-----+  
| StudentID | Name  | Age  |  
+-----+-----+-----+  
|         101 | Rahul |    20 |  
|         103 | Amit  |    19 |  
|         104 | Sneha |    22 |  
+-----+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql> |
```

8. Identify violations of integrity constraints in a given table and correct them.

Incorrect SQL Query

```
INSERT INTO Student (StudentID, Name, Age)
VALUES (101, NULL, 20);
```

Error

- Name cannot be NULL if it has a NOT NULL constraint.

Correct SQL Query

```
INSERT INTO Student (StudentID, Name, Age)
VALUES (101, 'John', 20);
```

Explanation

- NOT NULL constraints require a valid value for the column.

Output:

```
mysql> CREATE TABLE Student (
->     StudentID INT PRIMARY KEY,
->     Name VARCHAR(50) NOT NULL,
->     Age INT
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Student (StudentID, Name, Age)
-> VALUES (101, 'John', 20);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Student;
+-----+-----+-----+
| StudentID | Name | Age |
+-----+-----+-----+
|      101 | John |  20 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

9. Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.

Incorrect Table (Violates 1NF)

StudentID	Name	Subjects
101	John	Math, Science

Error

- Subjects contains multiple values in one column.

Correct Design

Student Table

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50)  
);
```

StudentSubjects Table

```
CREATE TABLE StudentSubjects (  
    StudentID INT,  
    Subject VARCHAR(50)  
);
```

Explanation

- 1NF requires each column to contain only atomic (single) values.

Output

```
mysql> CREATE TABLE Student (  
-> StudentID INT PRIMARY KEY,  
-> Name VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.05 sec)  
  
mysql> CREATE TABLE StudentSubjects (  
-> StudentID INT,  
-> Subject VARCHAR(50),  
-> FOREIGN KEY (StudentID) REFERENCES Student(StudentID)  
-> );  
Query OK, 0 rows affected (0.10 sec)  
  
mysql> INSERT INTO Student  
-> VALUES  
-> (101, 'John'),  
-> (102, 'Alice');  
Query OK, 2 rows affected (0.01 sec)  
Records: 2 Duplicates: 0 Warnings: 0  
  
mysql> INSERT INTO StudentSubjects  
-> VALUES  
-> (101, 'Math'),  
-> (101, 'Science'),  
-> (102, 'English'),  
-> (102, 'Physics');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4 Duplicates: 0 Warnings: 0  
  
mysql> SELECT  
-> s.StudentID,  
-> s.Name,  
-> ss.Subject  
-> FROM Student s  
-> JOIN StudentSubjects ss  
-> ON s.StudentID = ss.StudentID;  
+-----+-----+-----+  
| StudentID | Name | Subject |  
+-----+-----+-----+  
| 101 | John | Math |  
| 101 | John | Science |  
| 102 | Alice | English |  
| 102 | Alice | Physics |  
+-----+-----+-----+  
4 rows in set (0.00 sec)
```


10. Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.

Unnormalized Relation

StudentID	StudentName	DepartmentID
101	John	1
102	Alice	1

Problem

- DepartmentName is repeated for every student, causing redundancy.

Normalized Tables

Department Table

```
CREATE TABLE Department (  
    DepartmentID INT PRIMARY KEY,  
    DepartmentName VARCHAR(50)  
);
```

Student Table

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(50),  
    DepartmentID INT,  
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)  
);
```

Explanation

- Store department details in a separate table.
- Use a foreign key to link students with departments.
- This removes redundancy and satisfies 3NF.

Output

```
MySQL 8.0 Command Line Cli x + v
mysql> CREATE TABLE StudentDetails (
  -> StudentID INT,
  -> StudentName VARCHAR(50),
  -> DepartmentID INT,
  -> DepartmentName VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO StudentDetails
  -> VALUES
  -> (101, 'John', 1, 'CSE'),
  -> (102, 'Alice', 1, 'CSE'),
  -> (103, 'Bob', 2, 'ECE'),
  -> (104, 'David', 2, 'ECE');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM StudentDetails;
+-----+-----+-----+-----+
| StudentID | StudentName | DepartmentID | DepartmentName |
+-----+-----+-----+-----+
| 101 | John | 1 | CSE |
| 102 | Alice | 1 | CSE |
| 103 | Bob | 2 | ECE |
| 104 | David | 2 | ECE |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> CREATE TABLE Department (
  -> DepartmentID INT PRIMARY KEY,
  -> DepartmentName VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.04 sec)
```

```
MySQL 8.0 Command Line Cli x + v
mysql> CREATE TABLE Student (
  -> StudentID INT PRIMARY KEY,
  -> StudentName VARCHAR(50),
  -> DepartmentID INT,
  -> FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
  -> );
Query OK, 0 rows affected (0.09 sec)

mysql> INSERT INTO Department
  -> VALUES
  -> (1, 'CSE'),
  -> (2, 'ECE');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Student
  -> VALUES
  -> (101, 'John', 1),
  -> (102, 'Alice', 1),
  -> (103, 'Bob', 2),
  -> (104, 'David', 2);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT
  -> s.StudentID,
  -> s.StudentName,
  -> d.DepartmentName
  -> FROM Student s
  -> JOIN Department d
  -> ON s.DepartmentID = d.DepartmentID;
+-----+-----+-----+
| StudentID | StudentName | DepartmentName |
+-----+-----+-----+
| 101 | John | CSE |
| 102 | Alice | CSE |
| 103 | Bob | ECE |
| 104 | David | ECE |
+-----+-----+-----+
4 rows in set (0.00 sec)
```